SC2002 - OBJECT ORIENTED DES & PROG

Group Assignment (FCSF)

Done By:

- CAI SHIJUN
- CHAMOLI PARTH KUMAR
- CHANDA NEEL AUPURUP
- CHRISTOPHER CHAN YAO ZU

GITHUB : https://github.com/p7rthh/FCSF-SC2002-Group-Assignment

Table Of Contents

Table Of Contents **2**
1. Introduction **4**
2. System Overview **4**
2.1. Package Structure 4
2.2. High-Level Architecture and System Flow 4
3. System Architecture **5**
3.1. Architectural Overview 5
3.2. UML Class Diagram 6
3.3. Key Abstractions and Responsibilities 7
3.4. Relationships and Design Structure 8
4. Design Considerations **9**
4.1. Abstraction and Encapsulation 9
4.2. Decoupled UI Design 9
4.3. Battle Data Separation 9
4.4. Action Handling 9
4.5. Level and Wave Design 9
4.6. Extensibility (Expansion) 10
4.7. Use of Design Patterns 10
4.8. Trade-offs 10
5. Principles **11**
5.1. Single Responsibility Principle (SRP) 11
5.2. Open-Closed Principle (OCP) 11
5.3. Liskov Substitution Principle (LSP) 11
5.4. Interface Segregation Principle (ISP) 12
5.5. Dependency Inversion Principle (DIP) 12
6. Design Patterns Used **13**
6.1. Strategy Pattern 13
6.2. Factory Pattern 13
6.3. Command Pattern 13
6.4. State Support via Context Objects 14
7. Design Chart Summary **14**
8. Sequence Diagram Explanation **17**
9. Addition Features **18**
9.1. Replay System 18

9.2. Cooldown Display Adjustments 18
9.3. Input Validation and Error Handling 18
9.4. Prototype Pattern for Enemy Cloning 18
9.5. Dual-Interface GameCLI Design 18
10. Reflection 19
10.1. Difficulties Encountered 19
10.2. Lessons Learned 19
10.3. Further Improvements 20
11. Implementation Highlights 20
11.1. Centralised Damage Calculation 20
11.2. Status Effect Lifecycle 20
11.3. Cooldown Management 20
11.4. Wizard Scaling Mechanic 21
11.5. Backup Enemy Spawn 21
11.6. Replay Feature 21
12. Declaration 21
12.1. Declaration of Original Work for SC2002/CE2002/CZ2002 Assignment 21

1. Introduction

In this project, we are tasked to implement a turn-based combat area type game using Java, adhering to the object-oriented design principles. The system allows the player to engage in combat with one or multiple enemies using a series of actions within a turn-based battle system.

2. System Overview

2.1. Package Structure

The codebase is organised into six domain packages and one entry-point class, reflecting the layered architecture required by the assignment:

Package	Responsibility
com.sc2002.arena	Entry point (GameApp)
com.sc2002.arena.cli	Boundary layer: GameCLI (I/O), BattleController
com.sc2002.arena.engine	Control layer: BattleEngine, strategies, context, results
com.sc2002.arena.combat	Entity layer: Combatant, Player, Enemy, Warrior, Wizard, Goblin, Wolf, Inventory
com.sc2002.arena.action	Action entity layer: Action interface and all concrete actions
com.sc2002.arena.effect	Status effects: StatusEffect hierarchy and all concrete effects
com.sc2002.arena.item	Items: Item abstract class, concrete items, ItemFactory
com.sc2002.arena.level	Level configuration: Level, Wave
com.sc2002.arena.shared	Shared types: enums, BattlePresenter interface, PlayerActionSource interface

2.2. High-Level Architecture and System Flow

The architecture follows a classic three-layer pattern:

- Boundary Layer (CLI): GameCLI handles all terminal I/O. It implements both BattlePresenter (output) and PlayerActionSource (input) so that the battle engine never references System.in or System.out directly.
- Control Layer (Engine): BattleEngine orchestrates the game loop, delegating turn ordering to TurnOrderStrategy and enemy AI to

EnemyActionStrategy. It depends only on abstractions (interfaces), never on concrete classes.

- Entity/Domain Layer: Combatant, Action, StatusEffect, Item, and Level form the game model with no UI dependencies.

The system operates in a turn-based combat model with progression done in rounds. In each round, all combatants take turns based on a turn order strategy we have implemented, which will prioritize combatants with the higher speed attribute to begin first.

During the gameplay:

- The player selects a character class, items, and difficulty level.
- The battle proceeds in rounds managed by the BattleEngine.
- Turn order is determined using TurnOrderStrategy.
- Player actions are retrieved via PlayerActionSource and executed.
- Enemy actions are determined using EnemyActionStrategy.
- Status effects are applied across turn and round boundaries.
- The battle ends when either all enemies are defeated or the player's HP reaches zero.

This flow aligns with the game requirements, ensuring the combatants acts per round and that the game always ends in a clear win or loss outcome.

3. System Architecture

3.1. Architectural Overview

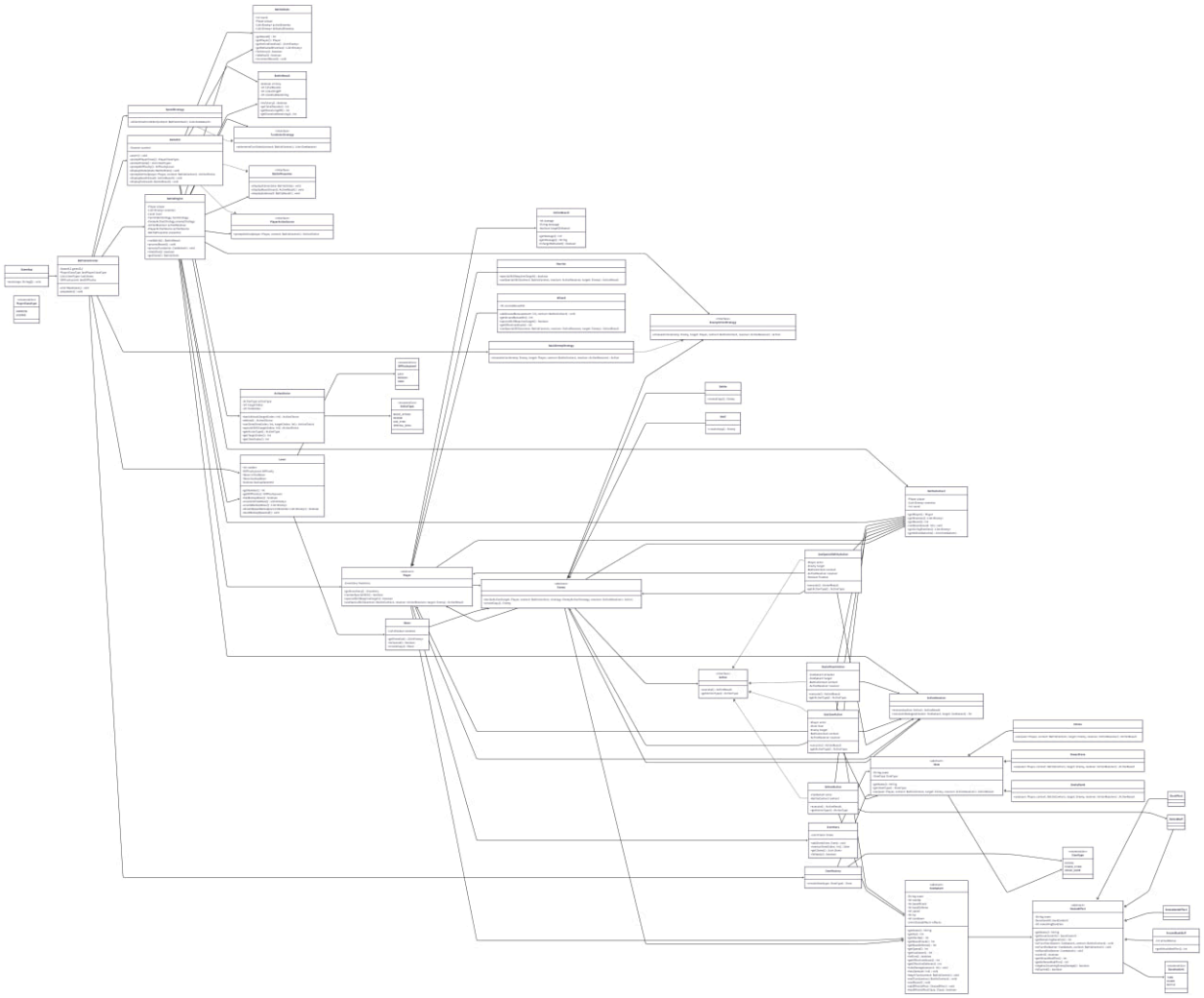
The system follows a layered architecture consisting of three main layers:

- Application Layer
  - GameApp: Entry point of the application.
  - BattleController: Handles game setup, replay functionality, and orchestration of game sessions.
- Boundary Layer (UI)
  - GameCLI: Handles all user input and output.
  - Implements:
  - BattlePresenter (output)
  - PlayerActionSource (input)
- Control Layer
  - BattleEngine: Core component responsible for managing rounds, turn execution, and victory conditions.

    - ActionResolver: Centralizes action execution and damage calculation.
- Domain Layer
    - Entities such as Combatant, Player, Enemy
    - Supporting components such as Inventory, Item, StatusEffect, Level, and Wave

This separation is to ensure that the user interface is decoupled from the game logic, allowing feasible maintenance and testing required by the assignment.

3.2. UML Class Diagram



The UML class diagram illustrates the relationships between the main components of the system, interaction, inheritance hierarchies, interfaces and associations.

3.3. Key Abstractions and Responsibilities

Component	Responsibilities
GameApp	Entry point of the program
BattleController	Manages game setup, replay, and engine initialization
GameCLI	Handles user interaction and displays battle information
BattleEngine	Controls battle flow, rounds, and turn execution
BattleContext	Provides runtime access to combatants and battle data
BattleState	Stores display-related state such as round and enemy status
ActionResolver	Executes actions and calculates damage
Combatant	Represents the model of all entities in battle, including shared attributes such as HP, attack, defense and speed.
Player	Extends Combatant, capable of user-controlled actions and inventory management.
Enemy	Extends Combatant and executes predefined attack actions.
Action	Encapsulates different combat actions such as attacking, defending and using skills.
StatusEffect	Represents effects that persist across multiple turns
Item	Defines consumable objects that affect gameplay
Level / Wave	Defines enemy configuration and spawning
TurnOrderStrategy	Determines the order which combatants act

These abstractions allow clear separation of concerns and improve system expansions capabilities.

3.4. Relationships and Design Structure

The system uses inheritance to model specialization, such as Player and Enemy classes extending the Combatant base class. This ensures shared functionality is centralized while allowing specific behaviors to be implemented in subclasses.

Composition is used for relationships such as:

- Combatant containing a list of StatusEffects
- Player maintaining an inventory of Items

Inheritance is used to model specialization (both Player and Enemy extend the Combatant base class, centralizing shared functionality while allowing subclass-specific behavior).

Quick Overview

Inheritance

- Combatant → Player, Enemy
- Player → Warrior, Wizard
- Enemy → Goblin, Wolf

Composition

- Player has an Inventory
- Combatant has a list of StatusEffect
- Level contains Wave

Dependency

- BattleEngine depends on:
    - TurnOrderStrategy
    - EnemyActionStrategy
    - BattlePresenter
    - PlayerActionSource
    - ActionResolver

These relationships enable flexibility and loose coupling between components.

4. Design Considerations

4.1. Abstraction and Encapsulation

The Combatant class is designed as an abstract class to encapsulate shared properties such as health points, attack, and defense. This avoids code duplication and ensures consistent behavior across all combat entities.

Encapsulation is applied by restricting direct access to internal attributes and exposing controlled methods such as takeDamage() and applyStatusEffect().

4.2. Decoupled UI Design

The UI is abstracted using:

- BattlePresenter for output
- PlayerActionSource for input

Although both are implemented by GameCLI, the BattleEngine depends only on these abstractions, ensuring UI independence.

4.3. Battle Data Separation

The system distinguishes between:

- BattleContext: runtime decision-making data
- BattleState: display and reporting data

This separation improves clarity and avoids mixing logic with presentation concerns.

4.4. Action Handling

Actions are encapsulated into separate classes (BasicAttackAction, DefendAction, etc.), allowing each action to manage its own execution logic independently.

The ActionResolver centralizes damage calculation and ensures consistency across all actions.

4.5. Level and Wave Design

Enemy configurations are encapsulated using:

- Level for difficulty setup

- Wave for grouping enemies

This allows flexible definition of initial and backup enemy spawns without hardcoding logic into the engine.

4.6. Extensibility (Expansion)

The system is designed to support future extensions without modifying core components. For example:

- New actions can be introduced by implementing the Action interface.
- New status effects can be added without modifying existing combat logic.
- Different turn order strategies can be implemented using the strategy pattern.

This design satisfies the requirement for extensibility outlined in the assignment.

4.7. Use of Design Patterns

Several design patterns are applied in this system:

- Strategy Pattern
  - Used for TurnOrderStrategy to allow different turn sequencing methods.
- Command Pattern
  - Implemented through the Action interface, where each action encapsulates its own execution logic.
- Factory-like Approach (Optional)
  - Used in creating combatants or items dynamically.

These patterns enhance flexibility and maintainability.

4.8. Trade-offs

While the use of abstraction and multiple interfaces improves extensibility, it increases the complexity of the system. This trade-off was necessary to meet the requirements for scalability and maintainability.
Additionally, separating logic into multiple classes increases the number of components, but results in better code organization and adherence to design principles.

5. Principles

5.1. Single Responsibility Principle (SRP)

The Single Responsibility Principle states that each class should have only one reason to change.

Each class has a well-defined responsibility:

- BattleController: game setup and replay
- BattleEngine: battle flow
- GameCLI: user interaction
- ActionResolver: combat logic
- Inventory: item management
- Item and StatusEffect classes: specific behaviours

This separation minimizes unintended side effects when modifying code.

5.2. Open-Closed Principle (OCP)

The Open-Closed Principle states that software entities should be open for extension but closed for modification.

The system supports extensibility through abstraction:

- New status effects can be added by extending StatusEffect
- New items can be added via Item
- New strategies can be added via TurnOrderStrategy and EnemyActionStrategy

However, adding new player action types requires modifications in multiple components (e.g., CLI and action mapping), indicating partial adherence.

5.3. Liskov Substitution Principle (LSP)

The Liskov Substitution Principle is upheld through the use of the Combatant abstraction.

- Both Player and Enemy extend Combatant and can be treated uniformly within the BattleEngine.
- The engine processes all entities using shared methods such as:
    - isAlive()

    - takeDamage()
    - beginTurn()
    - endTurn()

This allows the system to execute turns without needing to distinguish between different types of combatants.

Subclasses such as Warrior, Wizard, Goblin, and Wolf extend behavior while maintaining compatibility with the base class, ensuring correct polymorphic behavior.

5.4. Interface Segregation Principle (ISP)

The system applies ISP by using small, focused interfaces instead of large, general-purpose ones.

Examples include:

- BattlePresenter → only handles output
- PlayerActionSource → only handles input
- TurnOrderStrategy → only determines turn order
- EnemyActionStrategy → only determines enemy behaviour
- Action → only executes combat logic

This ensures that classes only depend on methods they actually use, reducing unnecessary coupling.

5.5. Dependency Inversion Principle (DIP)

The BattleEngine, as the core high-level module, depends on abstractions rather than concrete implementations.

It relies on:

- TurnOrderStrategy
- EnemyActionStrategy
- PlayerActionSource
- BattlePresenter

These abstractions are injected during initialization by BattleController.

This allows:

- UI to be replaced without changing the engine
- Different AI strategies to be plugged in
- Different turn-order logic to be swapped easily

This significantly reduces coupling and improves flexibility.

6. Design Patterns Used

Pattern	Where Applied	Benefit
Strategy	TurnOrderStrategy, EnemyActionStrategy	Swap algorithms at runtime without changing BattleEngine
Template Method	Combatant lifecycle hooks (beginTurn/endTurn/endRound)	Common structure, subclass-provided specifics
Factory Method	Level.fromDifficulty(), ItemFactory.create()	Centralised creation, decouples clients from constructors
Prototype	Enemy.createCopy()	Safe deep copy of enemy templates per game run
Command	Action interface (execute() → ActionResult)	Encapsulate requests as objects; decouples invoker from receiver
Observer (lightweight)	StatusEffect lifecycle hooks (onApply, onRoundEnd, expire)	Effects react to events without tight coupling to Combatant

6.1. Strategy Pattern

Used in:

- TurnOrderStrategy → SpeedStrategy
- EnemyActionStrategy → BasicEnemyStrategy

This allows behavior (turn order, enemy decisions) to be changed without modifying the engine.

6.2. Factory Pattern

Used in:

- ItemFactory

The factory creates items (Potion, PowerStone, SmokeBomb) based on ItemType, removing the need for direct instantiation in other classes.

6.3. Command Pattern

Implemented through:

- Action interface

- Concrete actions:
- BasicAttackAction
- DefendAction
- UseItemAction
- UseSpecialAbilityAction

Each action encapsulates:

- execution logic
- required data (actor, target, context)

This decouples action execution from decision-making.

6.4. State Support via Context Objects

Although not a formal GoF pattern, the system uses:

- BattleContext → runtime decision data
- BattleState → presentation state

This improves clarity and separates logic from display concerns.

7. Design Chart Summary

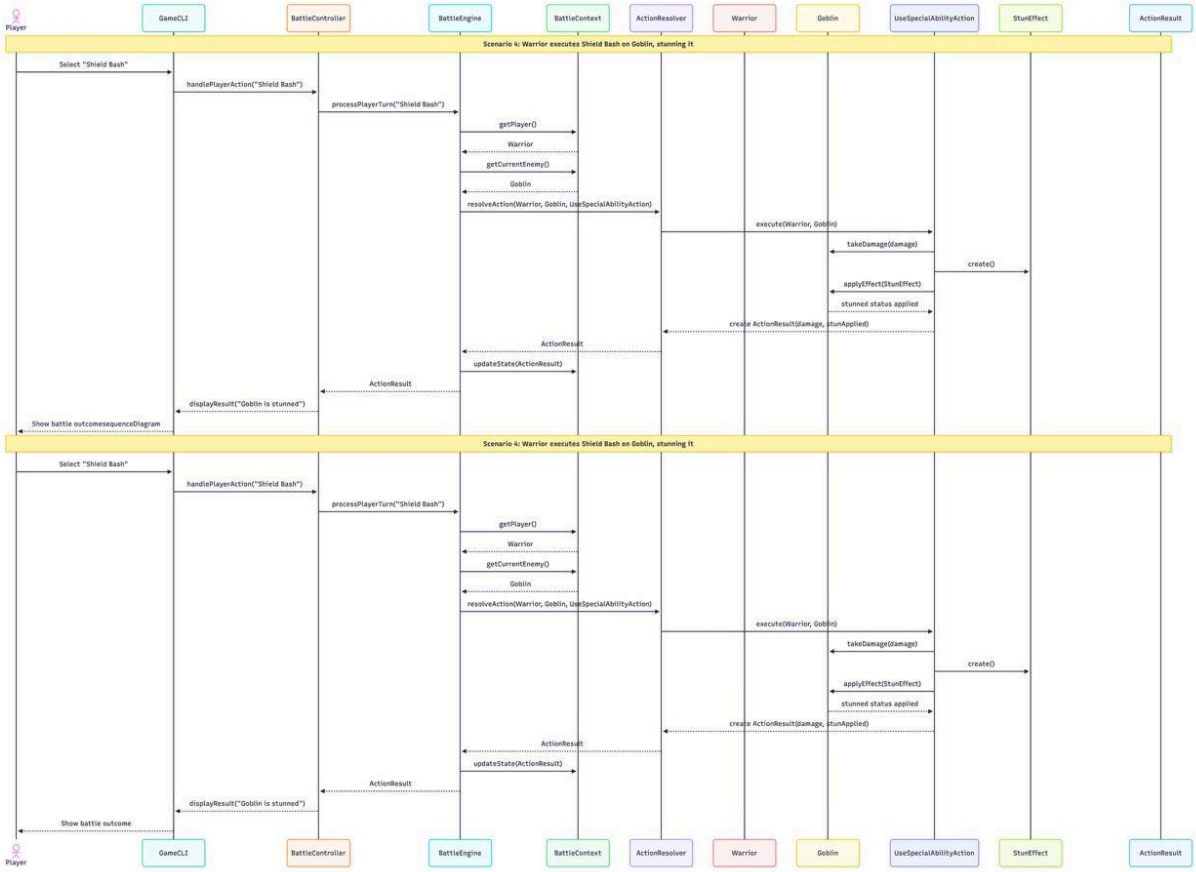
The following table summarises all key abstractions, their responsibilities, and the design principles they demonstrate:

Abstraction	Type	Responsibility	OO/SOLID Principles
Combatant	Abstract Class	Root character entity: HP, stats, effects, lifecycle	Encapsulation, Polymorphism, SRP, LSP
Player	Abstract Class	Player-specific: inventory, special skill contract	SRP, LSP, OCP
Enemy	Abstract Class	Enemy-specific: delegates action to strategy	SRP, LSP, DIP
Warrior / Wizard	Concrete Class	Implement useSpecialSkill()	Polymorphism, OCP
Goblin / Wolf	Concrete Class	Define stats and createCopy()	Prototype, OCP
Action	Interface	Encapsulate any in-turn action	ISP, OCP, Command pattern
BasicAttackAction	Concrete Class	Perform a basic attack	SRP, OCP
DefendAction	Concrete Class	Apply DefendBuff status effect	SRP, OCP

UseItemAction	Concrete Class	Delegate to Item.use()	SRP, OCP
UseSpecialAbilityAction	Concrete Class	Gate and execute special skill	SRP, OCP
StatusEffect	Abstract Class	Root effect entity: hooks and modifiers	Template Method, SRP, OCP
StunEffect	Concrete Class	Prevent action for 2 owner turns	SRP, OCP
DefendBuff	Concrete Class	Boost defense for 2 rounds	SRP, OCP
SmokeBombEffect	Concrete Class	Negate enemy damage for 2 rounds	SRP, OCP
ArcaneBlastBuff	Concrete Class	Permanent ATK bonus per Arcane kill	SRP, OCP
Item	Abstract Class	Root item entity: use() contract	SRP, OCP
Potion / SmokeBomb / PowerStone	Concrete Class	Item-specific use effects	SRP, OCP
ItemFactory	Concrete Class	Centralise item creation by type	Factory Method, SRP, DIP
TurnOrderStrategy	Interface	Determine combatant turn order	Strategy, OCP, DIP, ISP
SpeedStrategy	Concrete Class	Order by speed descending	Strategy
EnemyActionStrategy	Interface	Choose enemy's action	Strategy, OCP, DIP, ISP
BasicEnemyStrategy	Concrete Class	Always BasicAttack	Strategy, SRP
BattleEngine	Concrete Class	Orchestrate game loop	SRP, DIP, OCP
ActionResolver	Concrete Class	Execute actions and compute damage	SRP
BattleContext	Concrete Class	Read-only round snapshot for strategies	SRP, Encapsulation
BattleState	Concrete Class	Mutable game state record	SRP, Encapsulation

BattlePresenter	Interface	Display output contract for UI	ISP, DIP
PlayerActionSource	Interface	Input contract for player decisions	ISP, DIP
GameCLI	Concrete Class	CLI implementation of both UI interfaces	SRP, ISP
BattleController	Concrete Class	Wire all components; manage game flow	SRP, DIP
Level / Wave	Concrete Class	Enemy pool configuration per difficulty	Factory Method, Prototype, SRP
Inventory	Concrete Class	Manage player's item list	SRP, Encapsulation
DurationUnit	Enum	Classify effect timing scope	SRP
ActionType / ItemType / DifficultyLevel / PlayerClassType	Enums	Type-safe configuration constants	Encapsulation

8. Sequence Diagram Explanation



The sequence diagram models a combat scenario where interactions between the player, enemy, and system components are illustrated step-by-step.

The process includes:

- Player selecting an action through the UI
- Action being passed to the BattleEngine
- Execution of the action
- Application of status effects
- Enemy response
- Update of combat state

This demonstrates the interaction between boundary, control, and entity components.

9. Addition Features

9.1. Replay System

BattleController stores lastPlayerClassType, lastItems, and lastDifficulty after each game session. The post-game menu offers three options: replay with the same settings (no re-prompt), start a new game (full re-prompt), or exit. This improves user experience and demonstrates thoughtful state management.

9.2. Cooldown Display Adjustments

Player.getDisplayedCooldownAtTurnStart() returns max(0, cooldown - 1). This is a deliberate UX refinement: because beginTurn() decrements the cooldown before the player sees the menu, the raw cooldown value would be misleading. The adjusted display shows the cooldown as the player perceives it at decision time.

9.3. Input Validation and Error Handling

GameCLI.promptInt() wraps Scanner.nextInt() in a try-catch for InputMismatchException and loops until a valid integer in the specified range is entered. This prevents the game from crashing on bad input and avoids consuming excess tokens from the scanner's buffer.

9.4. Prototype Pattern for Enemy Cloning

Each Enemy subclass implements createCopy() returning a fresh instance via the Prototype design pattern. This ensures that Level.createInitialWave() and Level.createBackupWave() always return enemies with full HP regardless of how many times the level factory is called (e.g. replay). Goblin and Wolf delegate to their own constructors, keeping the copy logic trivially correct.

9.5. Dual-Interface GameCLI Design

GameCLI implements both BattlePresenter and PlayerActionSource in a single class, yet exposes them to BattleEngine as two separate interface references. This allows GameCLI to be split into two classes in the future (e.g. separating a display renderer from an input handler) without any change to BattleEngine.

10. Reflection

10.1. Difficulties Encountered

- Status effect timing was the most complex aspect of the implementation. The specification states that Stun lasts for the 'current turn and the next turn', while Defend and Smoke Bomb last for the 'current round and the next round'. These are fundamentally different timing units. The resolution was the DurationUnit enum (OWNER_TURN vs ROUND vs BATTLE), with separate decrement calls in endTurn() and endRound(). This took several iterations to get right.
- The Power Stone interacts with the special skill cooldown in a non-obvious way: it must trigger the skill effect without starting or advancing the cooldown. Initial designs routed through UseSpecialAbilityAction with a freeUse flag, but this polluted the action class with dual-mode logic. The final design has PowerStone call player.useSpecialSkill() directly, which is simpler and more correct.
- Backup spawning required careful sequencing: the backup wave must enter only after the current wave is fully cleared, and must enter at the start of the next round (not mid-turn). The shouldSpawnBackup() check placed at the end of processTurn() ensures this.

10.2. Lessons Learned

- Designing to interfaces first, BattlePresenter, PlayerActionSource, TurnOrderStrategy, EnemyActionStrategy before writing any implementation forced us to think about what each component truly needed from its collaborators. This discipline naturally produces low coupling.
- The SOLID principles are not independent rules but a system: SRP encourages small classes, which makes OCP and DIP easier; DIP requires interfaces, which enables OCP; ISP keeps interfaces focused, which supports SRP. Treating them as a unified philosophy rather than a checklist produced a more coherent architecture.
- Separating BattleContext (read-only snapshot for strategies) from BattleState (mutable game record for the engine and presenter) prevented subtle bugs where an effect or strategy could accidentally modify live game state.

10.3. Further Improvements

- A save/load system could persist BattleState to JSON, enabling the player to resume a session.
- A GUI layer (JavaFX or Swing) could be introduced by implementing BattlePresenter and PlayerActionSource with graphical components, with zero changes to the engine.
- Additional enemy types and difficulty levels can be added by extending the Enemy hierarchy and adding cases to Level.fromDifficulty()n and no other classes need modification.
- An ActionFactory could centralise action instantiation if the action-building logic in BattleEngine grows more complex, further reducing coupling between the engine and concrete action classes.

11. Implementation Highlights

11.1. Centralised Damage Calculation

Damage is handled in ActionResolver:

```
damage = max(0, attacker.getEffectiveAttack() - target.getEffectiveDefense())
```

Special logic:

- Smoke bomb negates enemy damage completely

11.2. Status Effect Lifecycle

Effects are applied through:

- beginTurn()
- endTurn()
- endRound()

Duration tracked using DurationUnit:

- TURN
- ROUND
- BATTLE

11.3. Cooldown Management

- Stored in Combatant
- Reduced in beginTurn()
- Reset when special ability is used
- PowerStone allows special skill usage without resetting cooldown

11.4. Wizard Scaling Mechanic

- Wizard has arcaneBonusAtk
- Gains cumulative attack boost
- Uses ArcaneBlastBuff for temporary scaling

11.5. Backup Enemy Spawn

Handled by:

- Level.shouldSpawnBackup()
- BattleEngine.checkBackupSpawn()

This allows dynamic enemy reinforcement.

11.6. Replay Feature

BattleController stores:

- last selected class
- items
- difficulty

Allows restarting with the same configuration.

The system implements key mechanics such as:

- Damage calculation using the formula:
- Damage = max(0, Attack - Defense)
- Status effect duration tracking across multiple turns
- Cooldown management for special skills
- Backup spawning of enemies upon elimination of initial wave

These features ensure compliance with the functional requirements of the assignment.

12. GitHub

Link: https://github.com/p7rthh/FCSF-SC2002-Group-Assignment

13. Declaration

13.1. Declaration of Original Work for SC2002/CE2002/CZ2002 Assignment

We hereby declare that the attached group assignment has been researched, undertaken,
completed, and submitted as a collective effort by the group members listed below.

We have honored the principles of academic integrity and have upheld Student Code of
Academic Conduct in the completion of this work.
We understand that if plagiarism is found in the assignment, then lower marks or no marks will be awarded for the assessed work. In addition, disciplinary actions may be taken.

Name	Course	Lab Group	Date
Christopher Chan Yao Zu	SC2002	FCSF	18/4/2026
CAI SHIJUN	SC2002	FCSF	18/4/2026
CHAMOLI PARTH KUMAR	SC2002	FCSF	18/4/2026
CHANDA NEEL AUPURUP	SC2002	FCSF	18/4/2026